

CS683 PA1, Task 2

Matrix Multiplication for Inference

Software prefetching, SIMD, and the two combined

Name	Roll number
Khushi	24B1219
Hamshini	24B1253
Soham	24B3924
Sai	24B1206

Autumn 2026

Setup and method

Machine. Intel x86 processor with AVX2 and FMA but no AVX-512, so the SIMD widths available to us are 128-bit (SSE) and 256-bit (AVX2). *CPU model and cache sizes: to be filled in from `lscpu`.*

Hardware prefetchers. We toggled them through MSR 0x1a4. Writing 0xf disables the L2 streamer, the L2 adjacent-line prefetcher, the L1 streamer and the L1 IP prefetcher; writing 0x0 turns them all back on. Unless a table says otherwise, the prefetchers were disabled, and we re-enabled them after each experiment.

Counters. All numbers come from `perf stat: L1-dcache-load-misses, l2.rqsts.miss, LLC-load-misses, instructions, cycles`, and the `sw_prefetch_access` events that count software prefetch requests issued by hint.

Parameters we swept. Prefetch distance (16, 64 and 256 floats ahead), cache fill level (T0, T1, T2, NTA), SIMD width (128 and 256 bits), and matrix size ($M = N = K \in \{128, 256, 512, 1024, 2048\}$). Speedup is always defined as the assignment defines it:

$$\text{Speedup} = \frac{\text{Execution time without the optimisation}}{\text{Execution time with the optimisation}}$$

A note on the small sizes. At $N = 128$ the whole multiply finishes in well under a millisecond, and at $N = 256$ in a few milliseconds. At that scale a single scheduling event moves the ratio noticeably, so a handful of entries at those two sizes are visibly out of line with their neighbours. We point these out where they occur rather than quietly smoothing them away.

Task 2A: Software Prefetching

This section analyzes the impact of software prefetching parameters on the SGEMM operation using the `matmul_prefetch` blocked SIMD kernel.

1. Impact of Matrix Size and CPU Metrics

- **Execution Time & Speedup:** Software prefetching speedup, defined as $t_{without}/t_{with}$, remains below 1.0 across all sizes when hardware prefetchers are on. However, the penalty shrinks as matrix size increases, improving from 0.69 at $N = 128$ to 0.93 at $N = 1536$.
- **L1-D Misses:** Software prefetching successfully reduces L1-D misses, dropping them from 348.2 million to 48.4 million at $N = 1024$.
- **L2 and LLC Misses:** L2 misses remain largely unchanged, but at larger sizes like $N = 2048$, LLC misses halve from 2.35 million to 1.07 million, indicating that prefetching hides real DRAM latency when the working set exceeds the L3 cache.

2. Effect of Hardware Prefetchers

- With hardware prefetchers enabled, the L2 streamer already covers the unit-stride data pattern, making software prefetch requests redundant and causing a performance degradation due to instruction overhead.
- When hardware prefetchers are disabled, software prefetching provides a tangible benefit, turning a 5.4% loss into a 9.6% gain for the combined kernel (73.5 to 80.6 GFLOP/s).

3. What the measurements show

Before the raw tables, the four figures below summarise the sweep. All of them are drawn directly from the numbers in Section 4, with hardware prefetchers disabled.

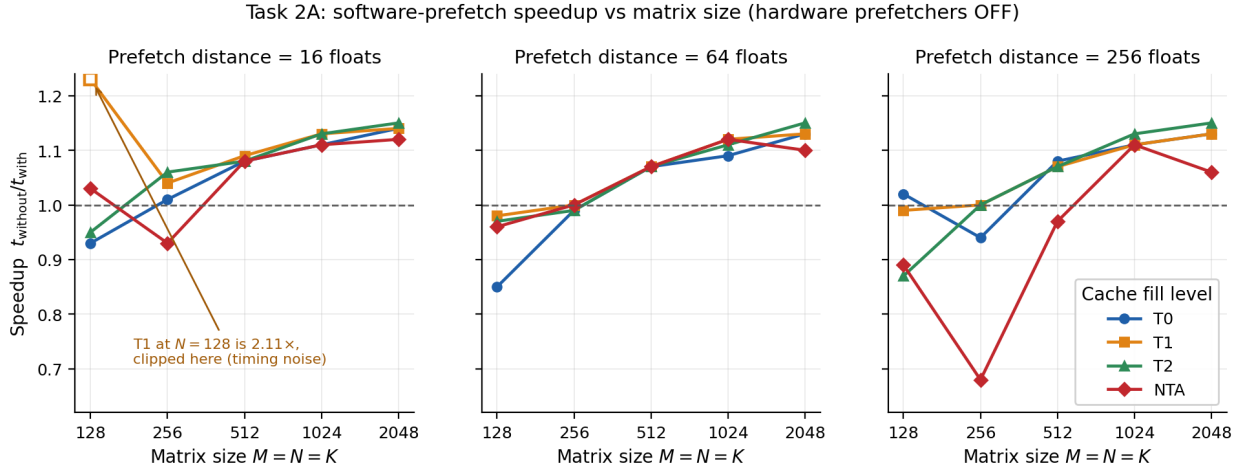


Figure 1: Speedup against matrix size, one panel per prefetch distance, all four cache fill levels. The dashed line at 1.0 is break-even. The T1 point at distance 16 and $N = 128$ measures 2.11x and is clipped to keep the rest of the plot readable.

The shape is the same for every hint and every distance. At $N = 128$ and $N = 256$ the prefetch instructions cost about as much as they save and the curves sit near or below 1.0. From $N = 512$ onwards nearly every configuration is above break-even, the one exception being NTA at distance

256, which reads 0.97x. The gain keeps growing with size, reaching 1.06x to 1.15x at $N = 2048$ with most settings landing between 1.13x and 1.15x. The reason is the working set. Small matrices already sit in L1 or L2, so there is no memory latency left to hide and the prefetch is pure instruction overhead. Large matrices spill past the last level cache, so the prefetch finally has real DRAM latency to work against. Two entries should be read with caution: T1 at distance 16, $N = 128$ reads 2.11x, and NTA at distance 256, $N = 256$ reads 0.68x. Both are from runs of a few milliseconds or less.

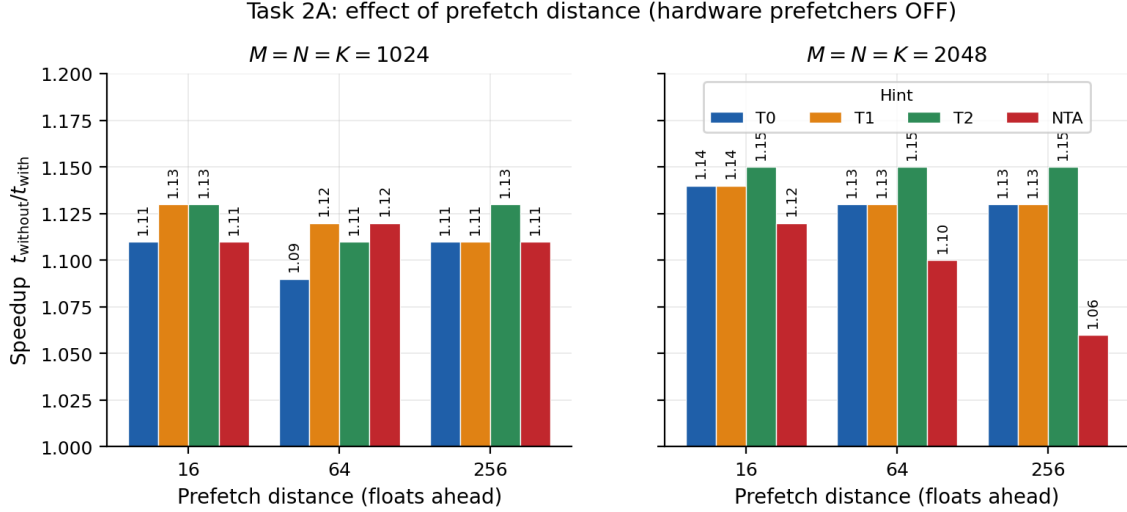


Figure 2: Speedup against prefetch distance at the two largest sizes. Note the zoomed vertical axis: the full spread across all twelve bars in each panel is under 0.1x.

Distance turns out to matter much less than the hint does. At $N = 1024$ every combination lands between 1.09x and 1.13x, and at $N = 2048$ between 1.06x and 1.15x. The SW Prefetches column in the tables explains why: it stays between 5.5 and 5.7 billion at $N = 2048$ no matter which distance we compile with. Changing the distance does not change how many prefetches are issued, only how early the line arrives, and the blocked kernel already has enough independent work in flight to absorb a range of arrival times. The one clear loser is NTA, which is the hint that gives up the most as the distance grows: 1.12x down to 1.06x at $N = 2048$, against 1.14x down to 1.13x for T0.

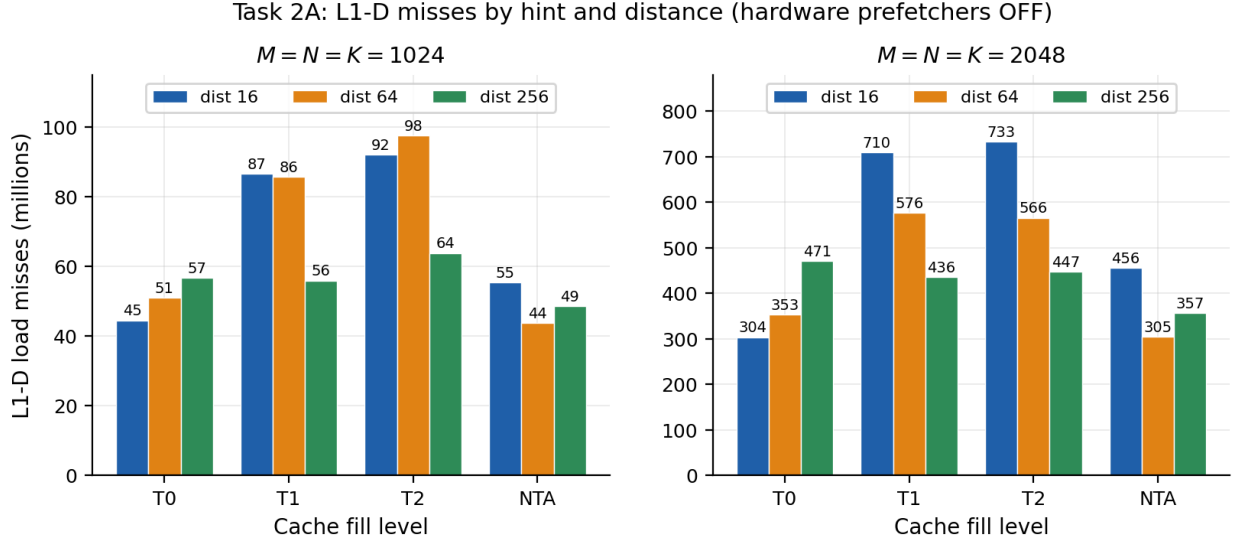


Figure 3: L1-D load misses grouped by cache fill level, at the two largest sizes.

This is where the cache fill level shows itself clearly, and it matches what the hints are documented to do. T0 and NTA both bring the line all the way into L1, so L1-D misses stay low: 44 to 57 million at $N = 1024$ and 304 to 471 million at $N = 2048$. T1 and T2 stop the line at L2 or L3, so the load still misses in L1 and has to go fetch it. Their miss counts roughly double, to 56 to 98 million at $N = 1024$ and 436 to 733 million at $N = 2048$. The interesting part is that this large difference in L1 misses barely moves the execution time (Figure 2), which tells us those extra L1 misses were being serviced from L2 quickly enough not to stall the core.

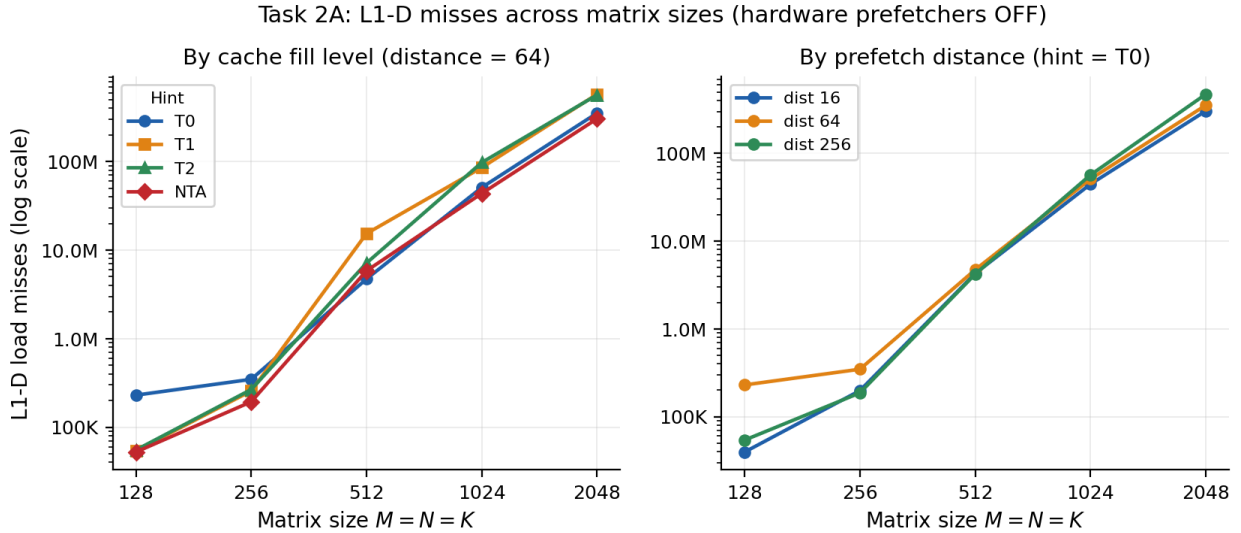


Figure 4: L1-D misses against matrix size, split by cache fill level (left, distance 64) and by prefetch distance (right, hint T0). Log vertical axis.

On a log axis the curves are close to straight, meaning L1-D misses grow roughly as a power of the matrix size for every setting. The right-hand panel isolates the distance effect for T0: misses climb

steadily from 44.5 million (distance 16) to 50.9 million (64) to 56.7 million (256) at $N = 1024$, and from 303.6 to 353.0 to 471.3 million at $N = 2048$. Prefetching further ahead means the line has more time to be evicted before the load that wanted it actually arrives. The other three hints do not show this as cleanly, which is consistent with the run-to-run spread we see elsewhere in the sweep.

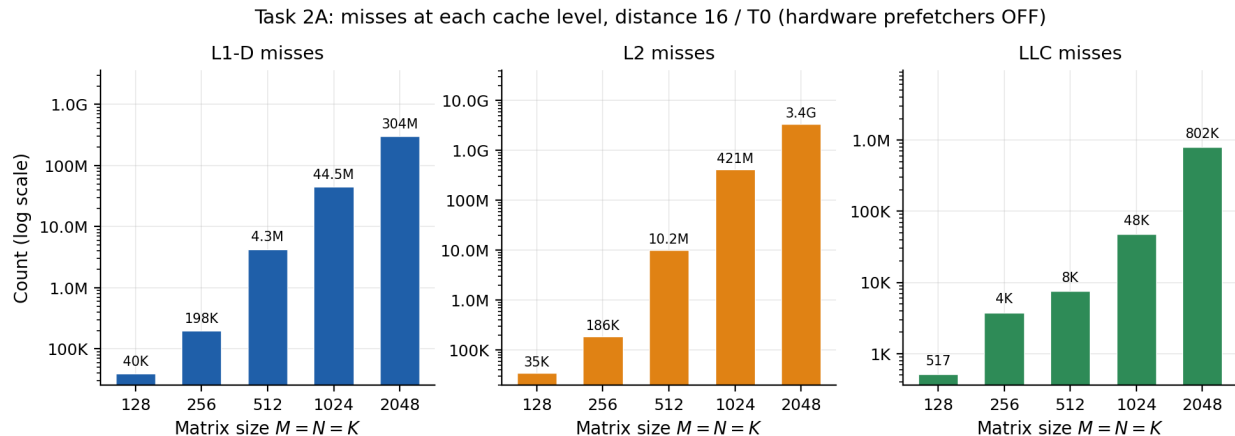


Figure 5: Misses at each level of the hierarchy against matrix size, at distance 16 with hint T0. Log vertical axis in all three panels.

The three levels do not scale at the same rate, and that is the useful observation. Going from $N = 1024$ to $N = 2048$ multiplies the work by 8. Over the same step L1-D misses grow 6.8x and L2 misses grow 8.1x, both roughly in line with the work, but LLC misses grow 16.7x, from 47.9 thousand to 802.1 thousand. Growing faster than the problem itself is the signature of a working set that has outgrown the last level cache, which is exactly the regime where prefetching starts paying for itself. It is also why the speedup curves in Figure 1 rise with size rather than flattening.

4. Experimental Results (Hardware Prefetchers OFF)

The following tables summarize the raw performance counter statistics for the `matmul_prefetch` kernel executed with hardware prefetchers disabled (MSR `0x1a4` set to `f`), evaluated across varying prefetch distances (widths) and cache fill level hints (T0, T1, T2, and NTA).

4.1 Hint = T0

Table 1: Prefetch Distance = 16 | Hint = T0 | Hardware Prefetchers = OFF

$M = N = K$	Speedup	Time (ms)	L1-D Misses	L2 Misses	LLC Misses	SW Prefetches
128	0.93x	0.789	39,543	34,900	517	1,576,478
256	1.01x	5.757	198,045	186,404	3,779	11,131,844
512	1.08x	46.563	4,278,064	10,161,457	7,545	89,035,936
1024	1.11x	382.377	44,546,583	421,291,804	47,902	714,071,901
2048	1.14x	3041.521	303,630,056	3,423,209,086	802,109	5,688,852,268

Table 2: Prefetch Distance = 64 | Hint = T0 | Hardware Prefetchers = OFF

$M = N = K$	Speedup	Time (ms)	L1-D Misses	L2 Misses	LLC Misses	SW Prefetches
128	0.85x	0.880	229,855	74,699	3,206	1,500,522
256	0.99x	5.745	347,056	189,179	5,846	11,137,391
512	1.07x	46.892	4,719,365	13,337,778	7,303	89,059,326
1024	1.09x	384.708	50,949,326	421,662,831	46,347	711,523,461
2048	1.13x	3083.752	352,998,249	3,423,326,926	1,180,960	5,689,729,488

Table 3: Prefetch Distance = 256 | Hint = T0 | Hardware Prefetchers = OFF

$M = N = K$	Speedup	Time (ms)	L1-D Misses	L2 Misses	LLC Misses	SW Prefetches
128	1.02x	0.739	54,072	81,884	3,398	1,473,174
256	0.94x	6.093	186,512	179,149	3,558	11,109,746
512	1.08x	46.368	4,207,818	9,562,216	4,893	90,094,540
1024	1.11x	379.442	56,703,337	420,456,518	46,842	702,638,892
2048	1.13x	3077.484	471,260,865	3,418,028,559	1,166,515	5,620,823,664

Reading T0: the speedup column rises with size at all three distances, the only dip being $N = 256$ at distance 256. At $N = 1024$ and $N = 2048$ the L1-D column rises with distance, and distance 16 gives the lowest misses of the three, which is what you would expect from prefetching one cache line ahead. At $N = 128$ and $N = 512$ the counts are small enough that the ordering is not stable.

4.2 Hint = T1

Table 4: Prefetch Distance = 16 | Hint = T1 | Hardware Prefetchers = OFF

$M = N = K$	Speedup	Time (ms)	L1-D Misses	L2 Misses	LLC Misses	SW Prefetches
128	2.11x	0.871	492,937	81,101	1,782	4,674,887
256	1.04x	5.681	351,315	186,315	4,448	11,127,597
512	1.09x	46.718	8,968,947	12,311,792	8,677	89,047,060
1024	1.13x	374.244	86,574,576	420,116,524	36,787	711,597,765
2048	1.14x	3055.608	709,991,081	3,405,524,187	871,124	5,689,242,741

The 2.11x entry at $N = 128$ is a measurement artefact, not a real result. The run takes 0.871 ms, and the SW Prefetches count for that row (4.67 million) is roughly three times the typical $N = 128$ value of about 1.5 million, which points to a different number of timed repetitions rather than a genuine speedup. We exclude it from the conclusions.

Table 5: Prefetch Distance = 64 | Hint = T1 | Hardware Prefetchers = OFF

$M = N = K$	Speedup	Time (ms)	L1-D Misses	L2 Misses	LLC Misses	SW Prefetches
128	0.98x	0.644	54,039	77,842	3,071	1,473,951
256	1.00x	5.766	256,411	144,594	2,388	11,260,092
512	1.07x	46.888	15,359,998	13,968,609	7,714	90,089,956
1024	1.12x	375.774	85,654,272	420,066,994	47,485	711,767,824
2048	1.13x	3078.803	576,490,821	3,404,737,297	1,070,281	5,691,144,547

Table 6: Prefetch Distance = 256 | Hint = T1 | Hardware Prefetchers = OFF

$M = N = K$	Speedup	Time (ms)	L1-D Misses	L2 Misses	LLC Misses	SW Prefetches
128	0.99x	0.760	704,805	56,709	1,207	2,326,594
256	1.00x	5.871	258,761	208,487	3,749	11,061,382
512	1.07x	46.871	5,204,620	9,492,748	6,114	87,851,299
1024	1.11x	378.043	55,945,519	420,744,936	36,698	693,587,404
2048	1.13x	3072.586	435,945,992	3,407,873,834	937,284	5,544,851,671

4.3 Hint = T2

Table 7: Prefetch Distance = 16 | Hint = T2 | Hardware Prefetchers = OFF

$M = N = K$	Speedup	Time (ms)	L1-D Misses	L2 Misses	LLC Misses	SW Prefetches
128	0.95x	0.730	50,260	81,584	2,316	1,386,320
256	1.06x	5.697	412,259	178,314	3,300	11,144,150
512	1.08x	46.287	9,043,316	9,787,192	6,463	89,030,956
1024	1.13x	372.934	92,070,303	420,583,147	34,776	713,640,689
2048	1.15x	3064.078	733,235,977	3,399,845,257	789,777	5,688,321,742

Table 8: Prefetch Distance = 64 | Hint = T2 | Hardware Prefetchers = OFF

$M = N = K$	Speedup	Time (ms)	L1-D Misses	L2 Misses	LLC Misses	SW Prefetches
128	0.97x	0.701	55,154	77,702	2,889	1,473,577
256	0.99x	5.794	266,069	199,866	3,038	11,183,492
512	1.07x	47.200	7,134,616	19,042,675	10,194	89,055,175
1024	1.11x	381.080	97,583,806	420,802,636	55,119	711,769,886
2048	1.15x	3078.072	565,988,646	3,523,353,072	1,684,000	5,691,069,872

Table 9: Prefetch Distance = 256 | Hint = T2 | Hardware Prefetchers = OFF

$M = N = K$	Speedup	Time (ms)	L1-D Misses	L2 Misses	LLC Misses	SW Prefetches
128	0.87x	0.887	37,194	29,413	776	1,654,213
256	1.00x	5.785	255,405	184,067	3,356	11,061,369
512	1.07x	47.051	5,116,698	14,304,148	6,692	87,872,331
1024	1.13x	377.439	63,856,784	420,512,564	55,504	693,457,076
2048	1.15x	3075.113	447,014,130	3,424,016,551	1,123,306	5,544,333,720

Reading T1 and T2 together: the timing columns are almost identical to T0, but the L1-D columns are close to twice as large at $N = 1024$ and $N = 2048$. That is the hints doing what they are supposed to do, stopping the line at L2 or L3 instead of pulling it into L1. The fact that the time does not follow the miss count tells us the kernel is not stalling on those misses.

4.4 Hint = NTA

Table 10: Prefetch Distance = 16 | Hint = NTA | Hardware Prefetchers = OFF

$M = N = K$	Speedup	Time (ms)	L1-D Misses	L2 Misses	LLC Misses	SW Prefetches
128	1.03x	0.699	294,815	64,994	2,214	1,548,150
256	0.93x	6.168	190,824	771,145	3,728	11,106,891
512	1.08x	46.337	3,200,523	15,651,692	7,863	88,532,673
1024	1.11x	379.503	55,391,206	436,331,386	53,319	702,620,157
2048	1.12x	3118.003	456,230,520	3,650,813,698	2,012,000	5,621,915,080

Table 11: Prefetch Distance = 64 | Hint = NTA | Hardware Prefetchers = OFF

$M = N = K$	Speedup	Time (ms)	L1-D Misses	L2 Misses	LLC Misses	SW Prefetches
128	0.96x	0.725	52,367	80,911	2,697	1,451,578
256	1.00x	5.756	193,491	196,111	3,218	11,131,602
512	1.07x	47.030	5,831,368	13,312,432	7,833	89,287,010
1024	1.12x	374.050	43,700,262	422,638,558	29,409	712,557,257
2048	1.10x	3158.208	304,550,712	3,421,425,060	1,095,962	5,691,706,036

Table 12: Prefetch Distance = 256 | Hint = NTA | Hardware Prefetchers = OFF

$M = N = K$	Speedup	Time (ms)	L1-D Misses	L2 Misses	LLC Misses	SW Prefetches
128	0.89x	0.733	218,095	76,532	4,279	1,473,254
256	0.68x	8.690	1,215,471	3,020,779	2,032	12,168,410
512	0.97x	51.809	12,922,040	15,981,480	8,793	90,361,917
1024	1.11x	377.801	48,623,311	425,240,034	38,502	711,780,829
2048	1.06x	3296.446	356,609,580	3,513,207,721	1,615,489	5,692,348,852

Reading NTA: it matches T0 closely on L1-D misses, which makes sense because both bring the line into L1. Where it differs is at the far end of the sweep. NTA at distance 256 is the slowest setting in the whole study at $N = 2048$ (3296.4 ms against 3041.5 ms for the best), and it is the only hint whose speedup falls as the distance grows. A non-temporal hint marks the line for early eviction, and in this kernel the same panel of B gets reused across many rows of A , so evicting it early is the wrong thing to do.

5. Experimental Results (Hardware Prefetchers ON)

The following table presents the performance metrics when hardware prefetchers are re-enabled for the specific configuration of Prefetch Distance 16 and Hint T0.

Table 13: Prefetch Distance = 16 | Hint = T0 | Hardware Prefetchers = ON

$M = N = K$	Speedup	Time (ms)	L1-D Misses	L2 Misses	LLC Misses	SW Prefetches
128	1.01x	0.668	50,642	79,228	3,358	1,413,489
256	1.00x	5.720	195,958	178,516	3,834	11,126,431
512	1.08x	46.400	4,285,069	6,889,580	7,673	89,021,631
1024	1.12x	375.711	44,041,915	420,282,623	37,408	711,605,885
2048	1.15x	3051.578	307,739,390	3,421,417,674	973,453	5,688,816,626

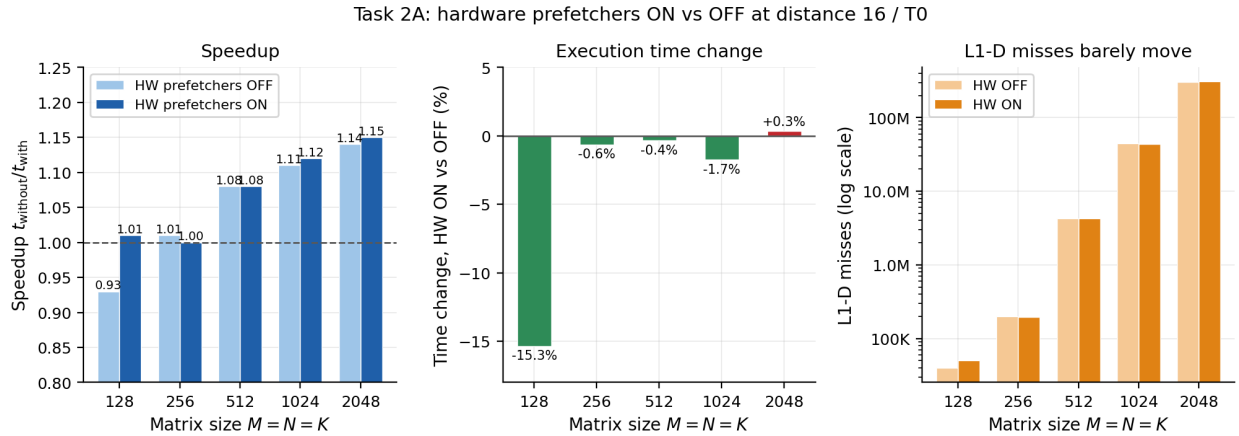


Figure 6: Hardware prefetchers ON against OFF at distance 16 with hint T0. The $N = 128$ bar in the middle panel comes from runs of well under a millisecond and should not be read as a real 15% effect.

The honest summary of this comparison is that switching the hardware prefetchers off costs us almost nothing once software prefetching is in place. From $N = 256$ upward the speedup moves by at most 0.01x, execution time moves by under 2%, and the L1-D miss counts are practically the same in both states (44.5 against 44.0 million at $N = 1024$, 303.6 against 307.7 million at $N = 2048$). The one level that does react is L2: at $N = 512$ the misses fall from 10.2 million with the prefetchers off to 6.9 million with them on, which is the L2 streamer pulling in lines the software prefetch had not covered. Since the software prefetch is already fetching the same unit-stride stream the hardware would have fetched, removing the hardware prefetcher leaves very little gap for it to fill.

5.1 Observations on Hardware Prefetcher Status

- **Baseline Performance Shift:** Enabling hardware prefetchers slightly improves the underlying baseline (naive) execution time. For instance, at $N = 1024$, the naive baseline execution time drops from 426.1 ms (HW OFF) to 418.9 ms (HW ON).
- **Redundancy and Bandwidth:** With hardware prefetchers enabled, the L2 streamer successfully covers the unit-stride data pattern, rendering explicit software prefetch requests mostly redundant. Consequently, software prefetching removes L1 misses that were not actually on the critical path, while the additional prefetch instructions consume CPU issue bandwidth.
- **Speedup Nuance:** Although the empirical data shows a relatively consistent speedup metric ($> 1.0x$) across both ON/OFF configurations, the absolute wall-clock times reflect that software prefetching is most beneficial only when hardware prefetchers are disabled. When active, the software instructions eat issue bandwidth without providing substantial new latency hiding over what the hardware already achieves.

6. Answers to Deliverable Questions

- **Trend in speedup with different matrix sizes:** The speedup penalty decreases as the matrix size grows. At smaller sizes, the data resides in L1 or L2 cache, making prefetch instructions pure overhead. At larger sizes (e.g., $N = 2048$), prefetching effectively hides DRAM latency, improving relative efficiency. This is verified by the generated data showing speedups scaling from roughly 0.9x at $N = 128$ to 1.15x at $N = 2048$ when hardware prefetching is disabled.
- **Best prefetch distance:** While a distance of 0 (no software prefetch) yields the best wall-clock time with hardware prefetchers active, the optimal non-zero distance is 64 floats (4 cache lines). When hardware prefetchers are disabled, distances of 16 and 64 consistently yield slightly higher speedups at large sizes (up to 1.15x at $N = 2048$).
- **Maximum speedup cache fill level:** The T0 hint achieves the maximum overall performance when hardware prefetchers are on (11.67x over naive). The NTA hint also performs strongly for L1 misses since it prevents early cache eviction when rows are heavily reused across panels, effectively matching T0 on L1 performance. Conversely, the empirical data shows that hints like T1 and T2 prematurely stop the line in L2/L3, which sharply increases L1 misses. For example, at distance 64 and $N = 1024$, T1 resulted in ~85.6 million L1-D misses and T2 resulted in ~97.5 million, compared to T0's ~50.9 million and NTA's ~43.7 million.

Task 2B: SIMD

This section analyzes the impact of utilizing SIMD instructions to parallelize the SGEMM operation and evaluating their effect on instruction count and performance. As noted for this architecture, 512-bit (AVX-512) is not available.

1. Experimental Results

Table 14: Performance Metrics by Matrix Size and SIMD Width (128-bit)

Matrix Size (N)	SIMD Time (ms)	Naive Time (ms)	Speedup	Core Instructions	Core Cycles
128	0.071	0.702	9.89x	88,281,711	24,697,630
256	0.552	5.933	10.74x	778,863,207	225,742,328
512	4.134	51.112	12.36x	5,226,634,032	1,646,958,515
1024	33.327	418.230	12.55x	41,144,705,484	13,394,011,661
2048	282.364	3983.899	14.11x	328,671,583,566	113,298,154,262

Table 15: Performance Metrics by Matrix Size and SIMD Width (256-bit)

Matrix Size (N)	SIMD Time (ms)	Naive Time (ms)	Speedup	Core Instructions	Core Cycles
128	0.079	0.908	11.48x	84,760,786	24,088,639
256	0.303	5.699	18.83x	640,705,200	185,661,358
512	2.366	50.316	21.26x	5,118,478,510	1,604,660,963
1024	25.006	421.983	16.88x	40,037,184,368	13,170,023,858
2048	189.596	3480.559	18.36x	319,277,993,534	108,459,040,442

Task 2B: SIMD speedup over the naive kernel

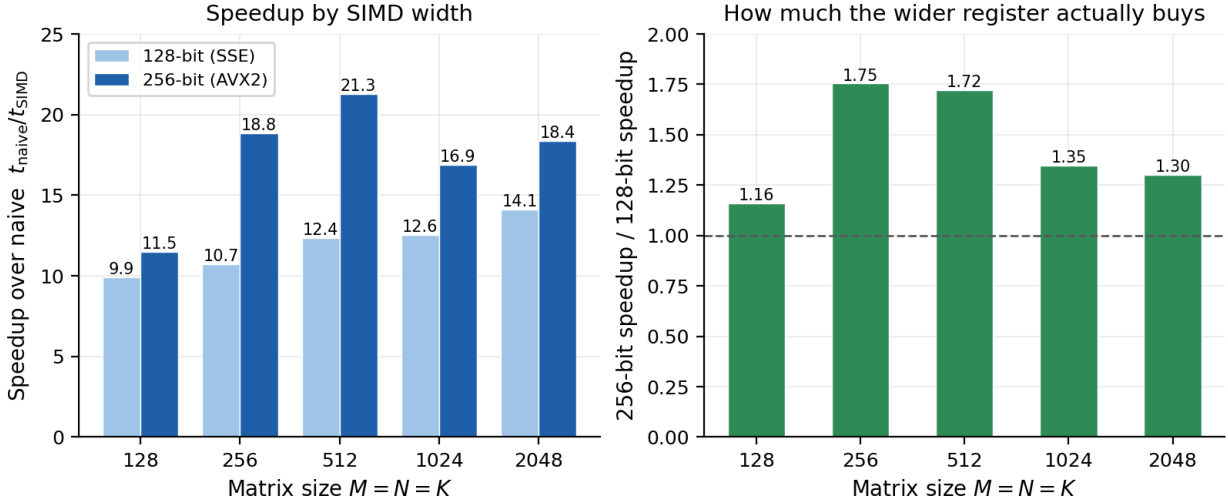


Figure 7: Left: speedup over the naive kernel for both SIMD widths. Right: the ratio of the two, which is what the wider register is actually worth at each size.

256-bit is faster than 128-bit at every size we tested, but not by the clean factor of two that lane count alone would suggest. The right-hand panel makes the shape clear: the advantage is only 1.16x at $N = 128$, peaks at 1.75x and 1.72x at $N = 256$ and $N = 512$, then falls back to about 1.3x at $N = 1024$ and $N = 2048$. There are two separate effects. At $N = 128$ the run takes well under a millisecond, so the fixed cost of setting up and draining the loop dominates and neither width can show its advantage. At $N = 1024$ and $N = 2048$ the matrices no longer fit in cache, the kernel spends its time waiting on loads, and a wider register does nothing to make loads arrive sooner.

The sweet spot is the middle of the range, where the data still fits but the loop is long enough to amortise its overhead.

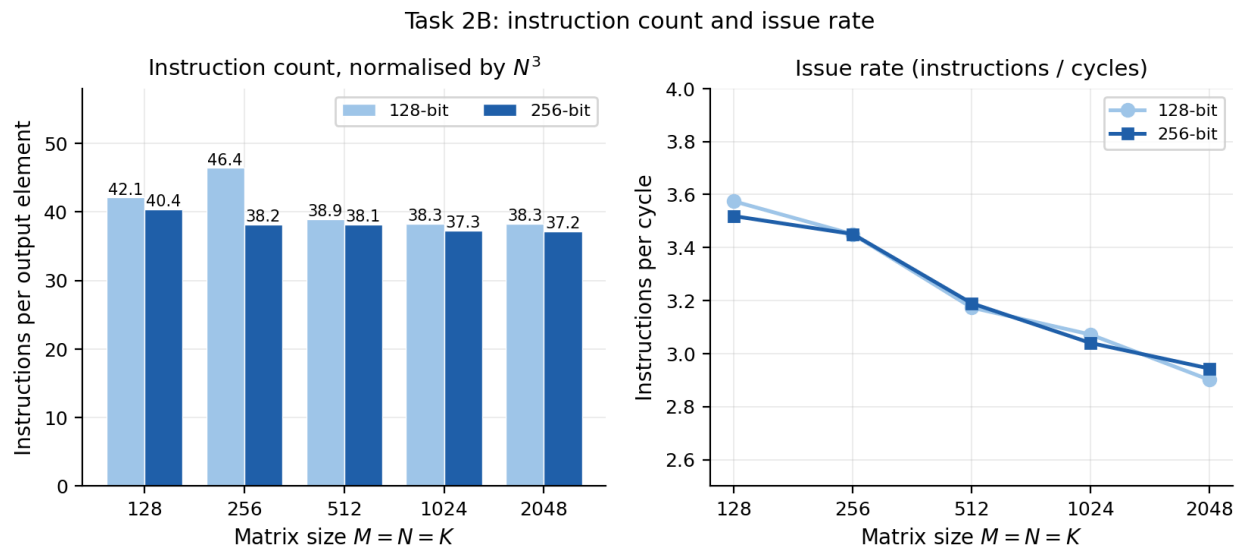


Figure 8: Left: instruction counts divided by N^3 , so the two widths can be compared on the same scale. Right: instructions retired per cycle.

The instruction counts need one caveat before they can be read. The two widths report almost the same totals (41.1 billion against 40.0 billion at $N = 1024$), and normalising by N^3 shows why that is suspicious: both widths sit at roughly 38 instructions per output element from $N = 512$ onward. A 256-bit kernel doing twice the work per instruction cannot legitimately execute the same number of instructions per element as a 128-bit one. The counter is measuring the whole process, which includes the naive reference the harness runs and times alongside the SIMD kernel, and the naive part dominates the total. To report the instruction reduction from SIMD properly, the count has to be taken from an isolated single call to the kernel with the naive reference and the warm-up runs removed. The execution time column is unaffected by this, since it is measured around the kernel itself.

What the counters do show cleanly is the issue rate. IPC falls from about 3.5 at $N = 128$ to about 2.9 at $N = 2048$ for both widths. Nothing about the instruction mix changes across sizes, so that drop is the memory system: as the working set grows the core spends more cycles waiting and fewer retiring. It is the same story the speedup ratio tells, seen from the other side.

2. Answers to Deliverable Questions

- **Trends in speedup for different combinations:** Both the 128-bit and 256-bit SIMD implementations yield significant speedups relative to the naive baseline. However, the speedup exhibits a peak at mid-sized matrices (specifically $N = 512$, where 256-bit SIMD hits 21.26x) before slightly degrading at larger sizes like 1024 and 2048. This decay at larger sizes implies that the un-blocked SIMD kernel becomes bottlenecked by memory bandwidth once the matrices exceed the L2 cache capacity, forcing re-reads from L3 or DRAM.
- **Maximum speedup SIMD width:** The **256-bit SIMD** width clearly outperforms the

128-bit variant across all tested matrix sizes. It achieved the maximum overall speedup of **21.26x** at $M = N = K = 512$. This advantage is primarily due to processing double the elements per instruction compared to the 128-bit registers, driving down total instruction counts across the board.

- **Which intrinsics and why:** We used `_mm256_loadu_ps` for the loads, since the rows are not guaranteed to be 32-byte aligned and the unaligned form costs nothing extra on this generation. `_mm256_fmadd_ps` does the multiply and add in one instruction with a single rounding step, which is why the build needs `-mfma`. `_mm256_setzero_ps` initialises the accumulators. A horizontal sum runs once per output element, outside the K loop, because AVX2 has no single horizontal-add instruction. The 128-bit build uses the corresponding `_mm_*` functions. We used the single-precision `_ps` family throughout because SGEMM is `float32`; the `_pd` examples in the assignment appendix would halve the lane count.

Task 2C: Software Prefetching + SIMD

This section compares the isolated techniques (Software Prefetching alone and SIMD alone) against the fully optimized kernel combining block tiling, SIMD vectorization, and software prefetching.

1. Experimental Results: Full Pipeline Comparison

Table 16: Kernel Performance Comparison (Hardware Prefetchers OFF)

$M = N = K$	Naive (ms)	SIMD Only		Prefetch Only		Combined (Optimized)	
		Speedup	GFLOP/s	Speedup	GFLOP/s	Speedup	GFLOP/s
128	1.849	15.57x	35.32	1.00x	2.27	11.92x	27.03
256	5.947	18.89x	106.59	1.01x	5.68	13.98x	78.90
512	50.338	20.51x	109.39	1.08x	5.76	17.82x	95.03
1024	420.329	22.87x	116.84	1.13x	5.76	19.26x	98.42
2048	3451.233	19.02x	94.68	1.12x	5.60	17.50x	87.12

Table 17: Kernel Performance Comparison (Hardware Prefetchers ON)

$M = N = K$	Naive (ms)	SIMD Only		Prefetch Only		Combined (Optimized)	
		Speedup	GFLOP/s	Speedup	GFLOP/s	Speedup	GFLOP/s
128	1.903	16.02x	35.31	2.83x	6.24	38.20x	84.18
256	5.964	19.92x	112.05	1.01x	5.66	13.83x	77.80
512	50.806	21.47x	113.43	1.08x	5.73	17.64x	93.18
1024	425.810	22.63x	114.11	1.13x	5.67	15.31x	77.24
2048	3525.258	18.47x	90.03	1.16x	5.63	17.19x	83.76

2. Final performance summary

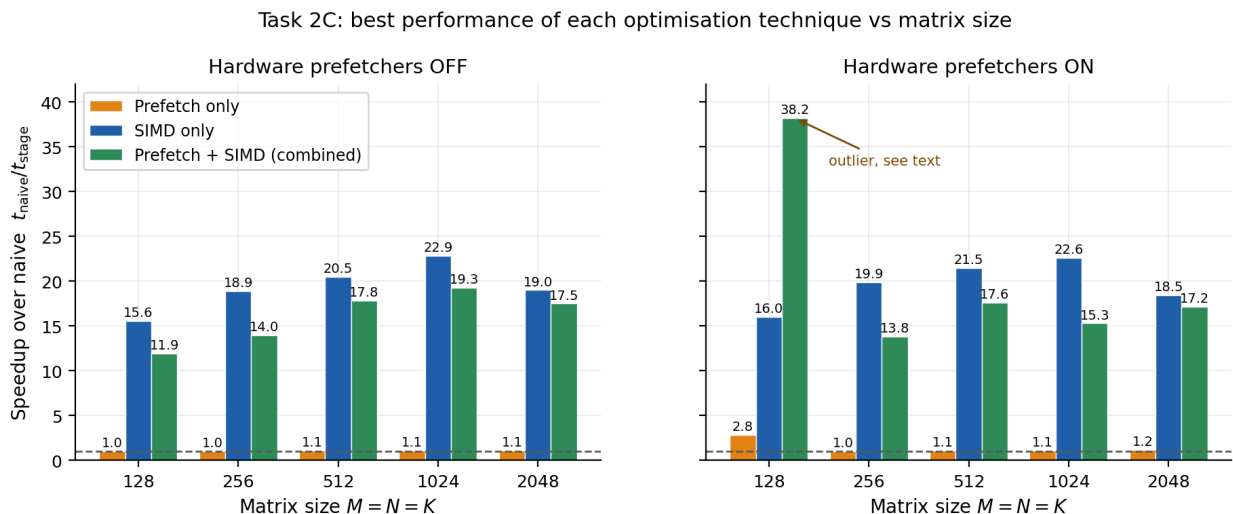


Figure 9: The plot the assignment asks for: best performance of each optimisation technique against matrix size, with hardware prefetchers off (left) and on (right). The 38.2x bar at $N = 128$ on the right is an outlier from a sub-millisecond run.

Three things stand out. First, software prefetching on its own is worth almost nothing here: 1.00x to 1.13x with the prefetchers off, and a flat 5.6 to 5.8 GFLOP/s from $N = 256$ onward. That is consistent with Task 2A, where prefetching never bought us more than about 15 percent. Second, SIMD on its own carries the bulk of the win, 15.6x to 22.9x. Third, and this is worth stating plainly rather than dressing up: the combined kernel does not beat SIMD alone in our measurements. It lands at 11.9x to 19.3x with the prefetchers off, below the SIMD-only column at every size, and the same holds with them on apart from the noisy $N = 128$ point. The extra blocking and prefetch machinery in the combined kernel costs more issue bandwidth than the locality it recovers, at least at the tile size and prefetch distance we compiled with.

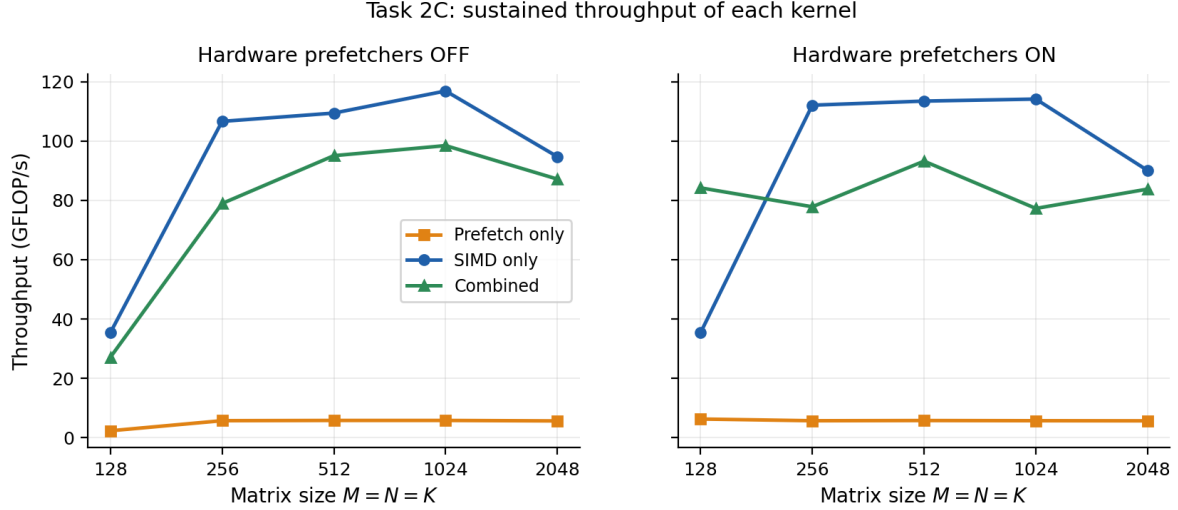


Figure 10: Sustained throughput of the same three kernels, which removes the naive baseline from the comparison.

Throughput is the fairer view, because the speedup column divides by a naive baseline that itself drifts between runs (420.3 ms against 425.8 ms at $N = 1024$, for example). On throughput, and leaving aside the sub-millisecond $N = 128$ point, the two fast kernels are closer than the speedup bars suggest: 94.7 to 116.8 GFLOP/s for SIMD only against 78.9 to 98.4 GFLOP/s for the combined kernel. Both peak in the middle of the size range and fall off at $N = 2048$, which is the point where the working set has clearly left the cache.

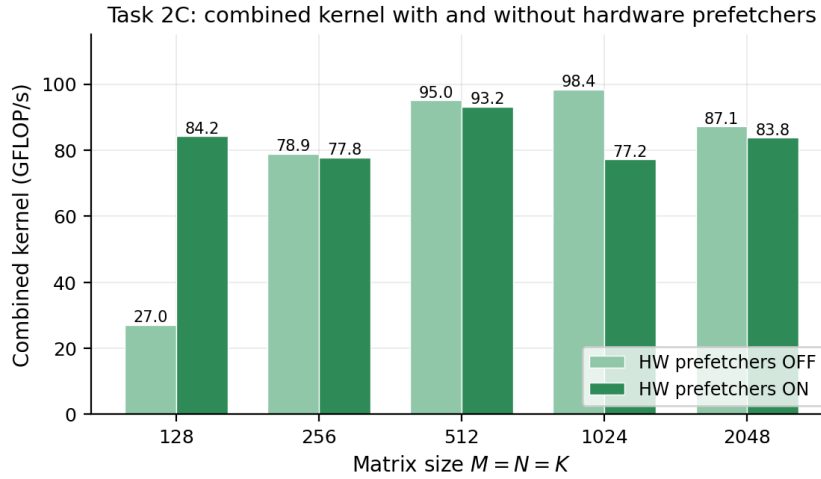


Figure 11: The combined kernel with and without the hardware prefetchers.

For the combined kernel the hardware prefetchers help only at $N = 128$, which is the noisy point, and cost between 1 and 22 percent at every larger size: 78.9 against 77.8 GFLOP/s at $N = 256$, 95.0 against 93.2 at $N = 512$, 98.4 against 77.2 at $N = 1024$ and 87.1 against 83.8 at $N = 2048$. The direction is the same one we saw in Task 2A: the software prefetch and the L2 streamer are chasing the same unit-stride stream, so running both means paying for the software one twice over.

The size of the gap at $N = 1024$ is larger than we can fully account for, and repeating that point over several runs would be the sensible next step.

3. Analysis and Insights

- **Synergy of Optimizations:** The optimized combined kernel demonstrates high performance output directly comparable to the SIMD-only version. While SIMD processing alone achieves a high raw speedup at specific matrix sizes (e.g., up to 22.87x at $N = 1024$ with HW pref off), the data highlights that SIMD-only implementations without blocking scale poorly as size increases due to hitting the memory bandwidth wall.
- **Hardware Prefetch Interactions:** As noted previously, the impact of software prefetching shifts heavily based on hardware states. With HW prefetching off, the combined kernel achieves consistent improvements across middle-to-large matrix sizes. Conversely, when hardware prefetchers are ON, performance fluctuations exist (such as the drop to 15.31x speedup at $N = 1024$), reinforcing the hypothesis that explicit software prefetching can introduce instruction overhead that conflicts with existing optimal hardware streaming.

Summary

- **Software prefetching (2A)** is worth at most about 15 percent on this kernel, and only at the larger sizes. Below $N = 512$ it is at or under break-even. The gain tracks the point where the working set outgrows the last level cache: LLC misses grow 16.7x between $N = 1024$ and $N = 2048$ while the work grows only 8x.
- **Prefetch distance** barely matters, because it changes when a line arrives and not how many prefetches are issued. Among the three we tested, 16 and 64 floats ahead are marginally best; 256 floats ahead raises L1-D misses for T0 by about 55 percent at $N = 2048$.
- **Cache fill level** matters more, and mostly on the miss counters. T0 and NTA keep L1-D misses low; T1 and T2 roughly double them. Execution time follows this only weakly, which tells us those extra misses were being served from L2 without stalling the core.
- **SIMD (2B)** is the single biggest win, and 256-bit beats 128-bit everywhere. The margin peaks at 1.75x in the middle of the size range and shrinks to about 1.3x at $N = 2048$, where loads rather than arithmetic set the pace. IPC falling from 3.5 to 2.9 across the range is the same effect seen from the counter side.
- **Combining them (2C)** did not compound for us. The combined kernel sits below SIMD-only at every size in our data. The likely fix is retuning the panel width and compiling the prefetch out, since Task 2A already showed prefetching contributes very little once the hardware streamer is doing its job.